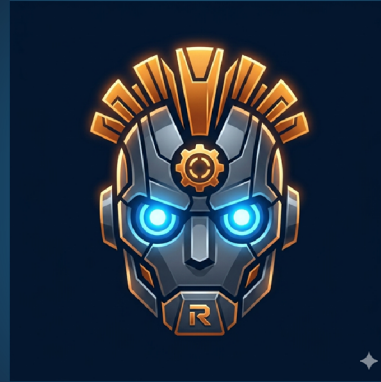




# Agentic Architecture in Practice

**Harness · LLM · Directives**

A working model for how agentic systems are designed, governed, and extended — with **RockBot** as the example.

Rockford Lhotka · [rockbot.dev](https://rockbot.dev) · [blog.lhotka.net](https://blog.lhotka.net)



THE THESIS

# What **is** an agent?

Not "an LLM with tools bolted on." In practice it's three distinct parts:



## **Harness**

The code that controls execution  
— the loop.

## **LLM**

Reasons and generates. A black  
box.

## **Directives**

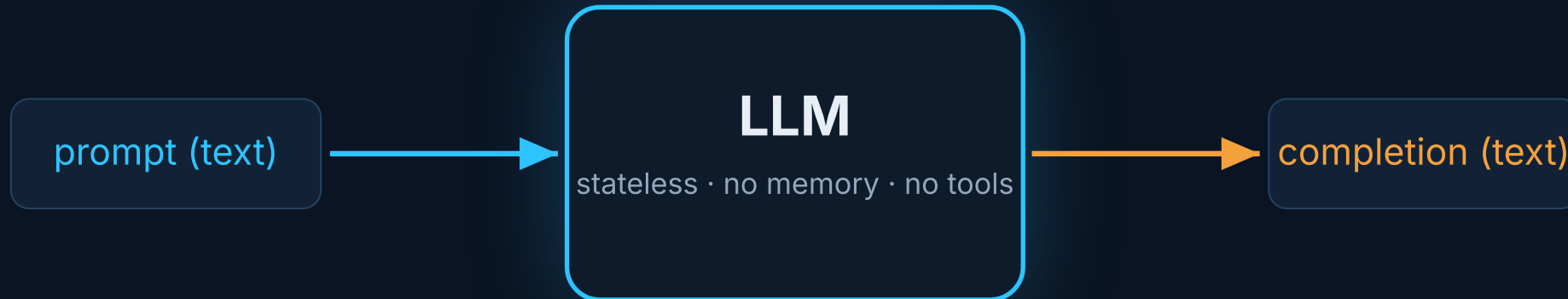
Shape behavior — scope, tone,  
rules.

Pull any one out and you don't have an agent.



# The LLM is a **black box**

It's a microservice: **text in** → **text out**. Nothing more.



It doesn't remember the last call.

It can't call a tool — it just **writes** that it wants to.

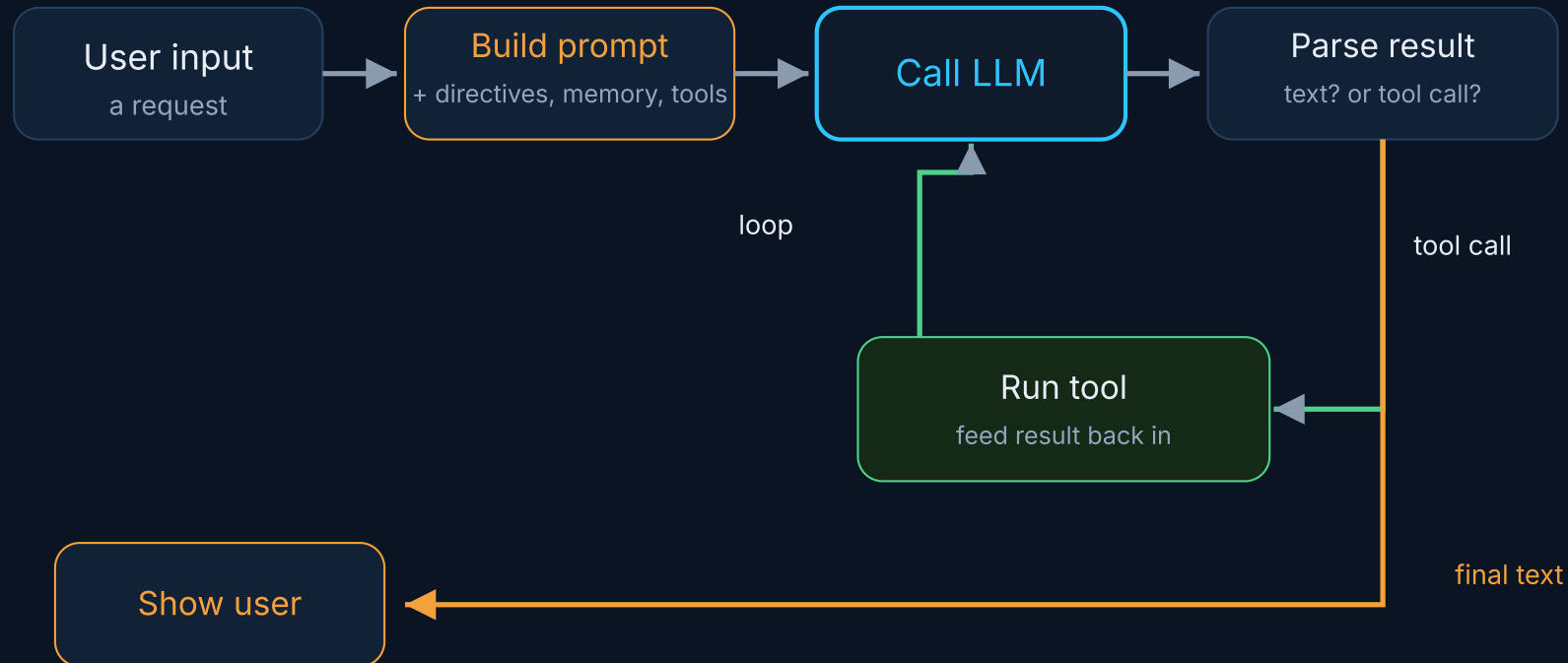
It takes no action. It only generates text.

So where does all the "agent" behavior come from? →

PART 2 OF 3 — THE HARNESS

# The harness is **the loop**

This is where the real engineering lives.



The **inner loop** — call a tool, feed the result back, call the LLM again — is what makes an agent feel agentic.

# Directives **shape behavior**

Scope · tone · rules-for-success. In RockBot, literally markdown files, composed into the system prompt.

 **soul.md**

**Scope & identity.**

Who the agent is, what it's for.

 **directives.md**

**Rules.**

Constraints that keep it safe and on-task.

 **style.md**

**Tone.**

How it communicates.

Layered & composable — base persona, then per-task, then per-skill:

soul + directives + style

+

task directive

+

skill

→

system prompt

Change behavior without changing code.

# What actually reaches the LLM

One ordered window, assembled each turn — and sequenced to protect the prompt cache:

▲ top · earliest tokens · the stable, cached prefix

**System prompt** soul · safety · directives · memory rules · style · active rules · guardrails cached ✓

**Conversation history** previous turns — stable prefix, reused unchanged cached ✓

— PROMPT-CACHE BOUNDARY · EVERYTHING BELOW IS REBUILT EVERY TURN —

**Date & time** minute-rounded, placed [here](#) so a fresh timestamp can't bust the cache per-turn

**Current user message** the new turn — what the model is answering user

**Recalled memory** long-term (BM25) · episodic · identity · knowledge graph per-turn · delta

**Skills · services · working memory**  
matched skills · A2A + MCP hints · scratch · patrol · subagent research per-turn

▼ newest tokens · the LLM generates from here

**Date/time sits below the cache boundary on purpose** — minute-rounded and after the stable prefix, so a per-turn timestamp doesn't bust the provider's prompt cache. Tool definitions (MCP · CLI · API) ride alongside in the request's tool list, not the messages.



THE CONCRETE EXAMPLE

## Meet RockBot

An **event-driven, process-isolated** autonomous agent framework for .NET.

"Nothing trusts  
the LLM."

message bus (RabbitMQ)

isolated processes

least privilege

MIT · open source

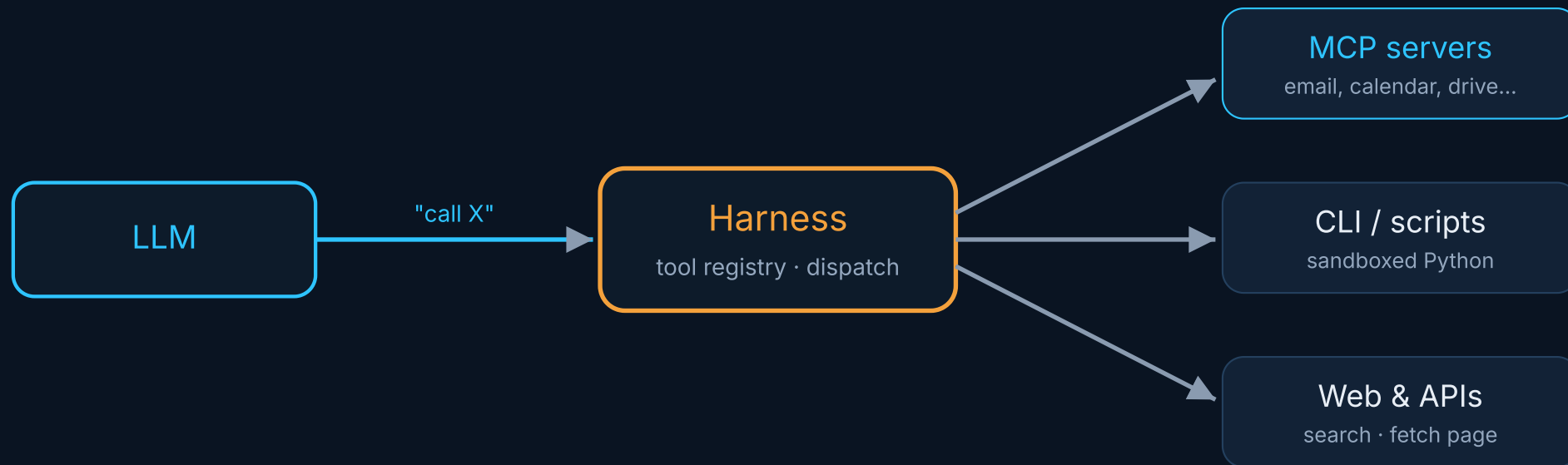
Everything that follows is a real subsystem in this codebase.



REACHING OUTWARD

## Tools: how the harness **acts**

The LLM only **asks** for a tool. The harness routes and runs it — then feeds the result back.



RockBot leans on **MCP** heavily — secrets live in the MCP servers, not the agent. (more on that later)

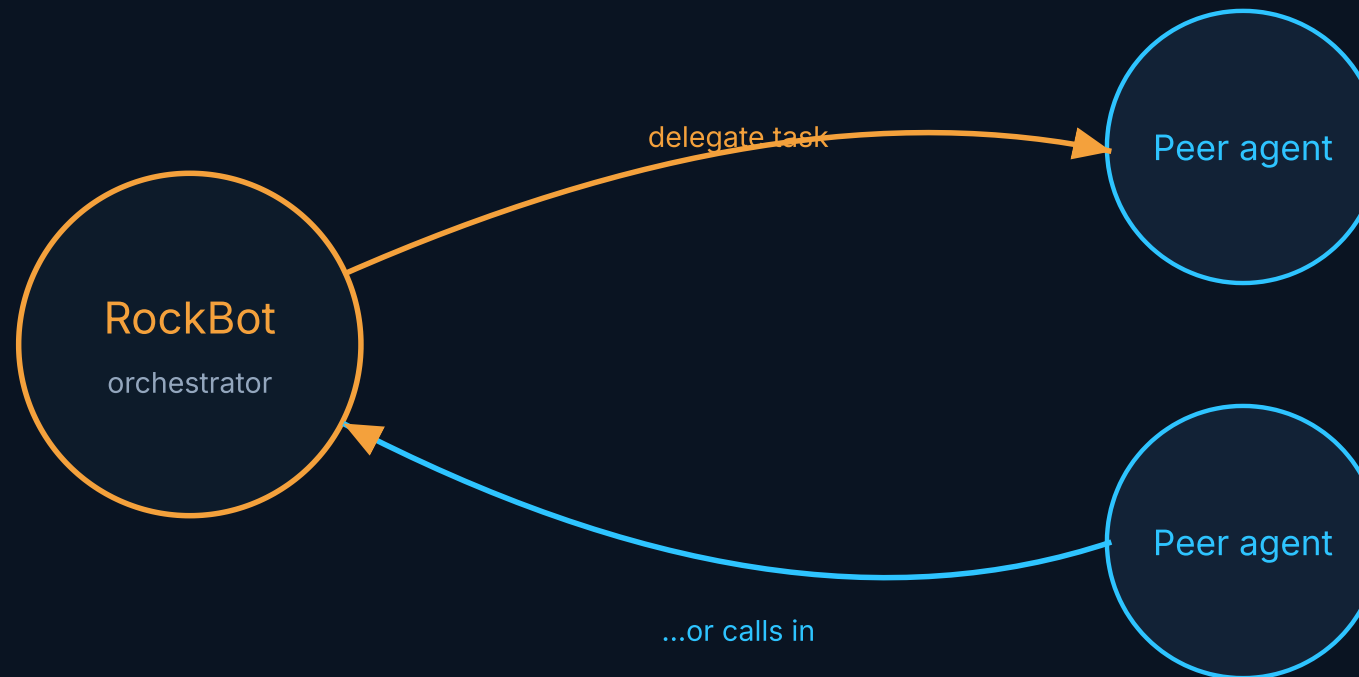




## REACHING OUTWARD

# Peers: agents calling agents

The **A2A** protocol lets RockBot delegate to other agents — or be delegated to.



A tool call reaches a service. An **A2A** call reaches a reasoning peer — its own harness, LLM, and directives.

## THE PIVOT

# The harness handles a lot

Once the core (loop + LLM + directives) is solid, this is what becomes possible:



The rest of the talk walks the spokes.

# Memory: three tiers + a graph



## Conversation

Last N turns. Session-scoped, ephemeral. The here-and-now.



## Working

Shared, namespaced scratch space with a TTL. Hand-off between sessions, tasks & subagents.



## Long-term

Permanent facts & preferences. Recalled per-turn by BM25 (+ vector) search.



## Knowledge graph

Entities & relationships for structured, relational reasoning.

The trick isn't storing it — it's **injecting only what's relevant**, every turn, without token bloat.

Delta injection: a memory recalled once isn't re-injected the same session.

# Skills: **fixed** guides & **evolving** know-how

Memory stores **facts**. Skills store **how to do something** — markdown procedures, recalled by BM25.

## **Subsystem guides**

Authored, fixed. "How to use the email MCP," "how working memory works." Ship with the agent.

## **Mutable skills**

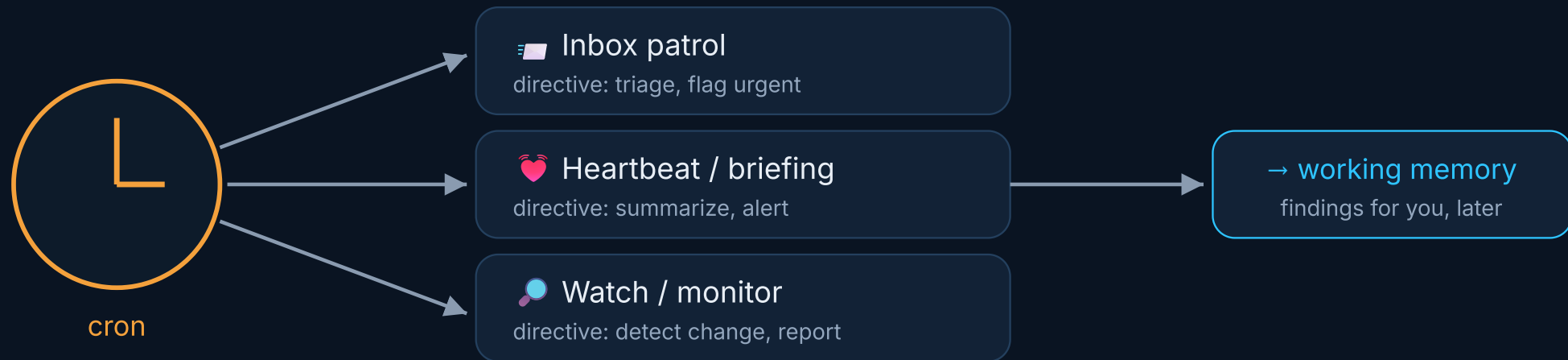
The agent **writes its own** — captures a procedure that worked, then refines it over time.



The refine step happens offline — in the dream. →

## Tasks on a schedule

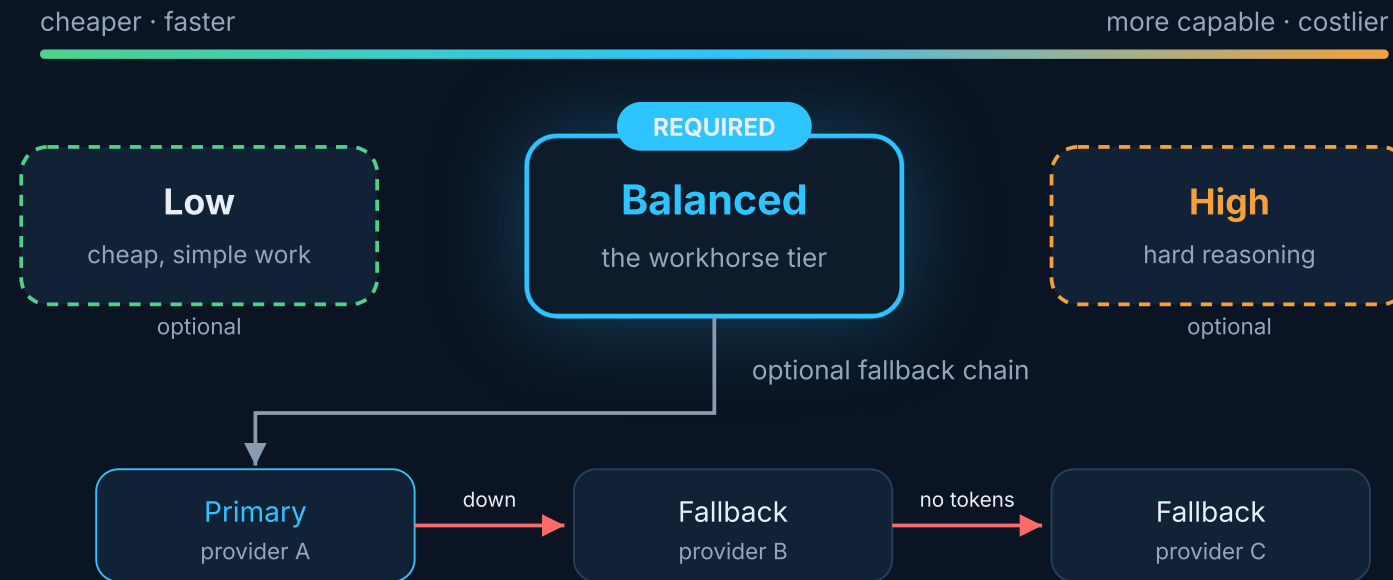
No user required. Cron-driven "patrol" tasks fire on their own — **each with its own directive.**



Same harness, same LLM — a **different directive** turns it into a different worker.

# LLM routing: tiers & fallback

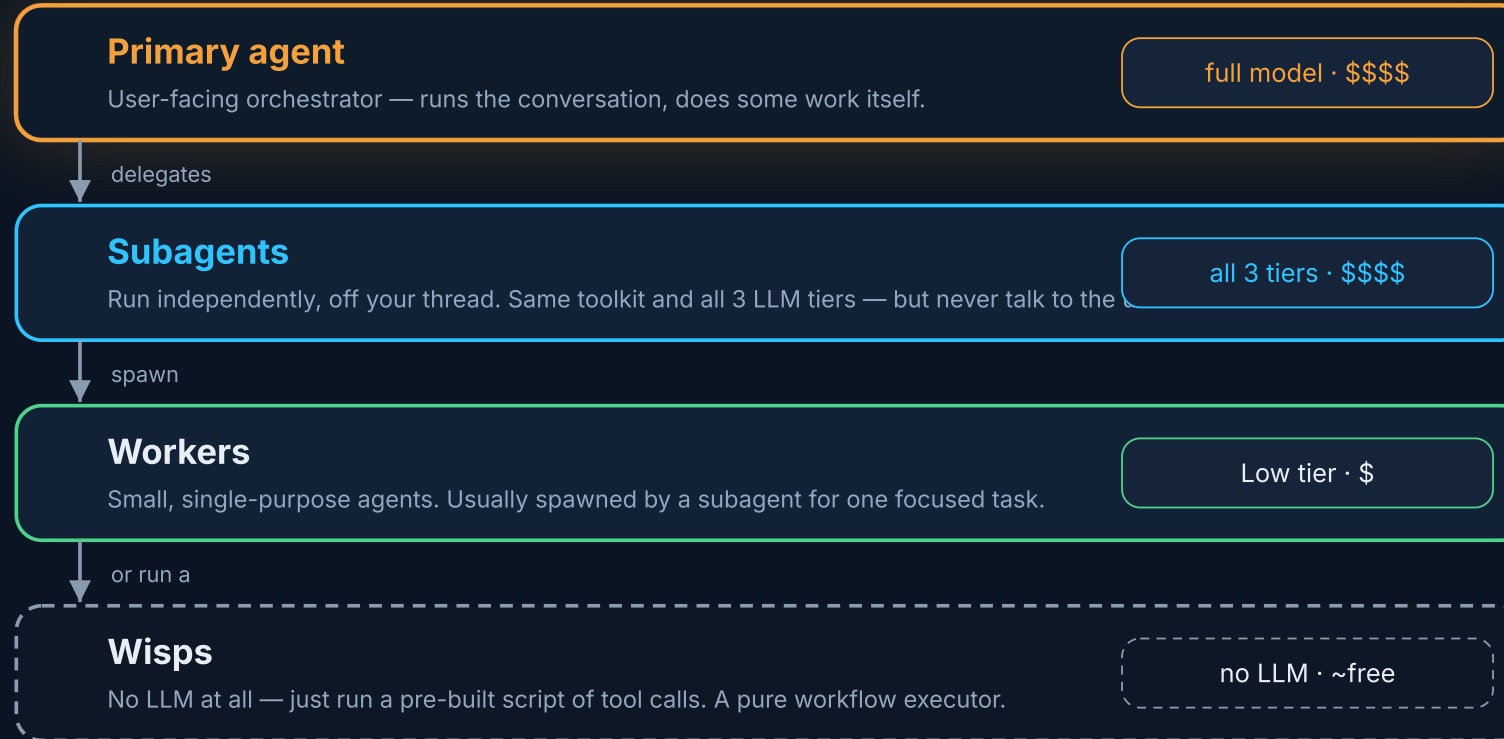
The LLM is swappable — so RockBot routes work across **cost/capability tiers**, and falls back when a provider fails.



**Balanced** is the one requirement. Low & High are levers for cost and capability — and the fallback chain keeps the agent working when a provider is down or out of tokens.

# The orchestrator delegates **down**

The primary agent runs the conversation — and pushes each task to the **lightest executor that can do it**. Routine work drops to a cheap model, or no model at all.



Primary and subagents both reach for any tier. The savings come below: **workers** on the Low tier, **wisps** with no model at all.

# RockBot dreams

A background cycle that runs offline and **refactors the agent's own knowledge** — no user, no goals changed.

## Consolidate memory

Merge duplicates, decay stale facts, mine anti-patterns from corrections.

## Optimize skills

Refine, merge, and cluster the skills it has written.

## Tune LLM routing

Learn which work belongs in which tier — Low, Balanced, or High — for the best cost/quality.

## Infer preferences

Notice patterns across conversations; learn how you like to work.

Yesterday's experience makes tomorrow's agent sharper — automatically.



# Principle of least privilege

The core philosophy, restated: **nothing trusts the LLM**. The core agent holds **no keys, no passwords**.



A compromised prompt can't leak a secret the agent never had.

# Untrusted code runs **sandboxed**

RockBot can run Python and pull web pages — but never in-process.

## **Python execution**

Runs in an **ephemeral, low-privilege container**. Spun up per run, destroyed after. No host access, no secrets, minimal blast radius.

## **Web search & fetch**

Search the web and retrieve pages — but that content is **untrusted input**, handled with the same suspicion as the LLM's output.

Same pattern everywhere: **assume the LLM (and the web) can be wrong or hostile** — and contain it.

## THE WORKING MODEL



Get the core right, and it extends:

memory

skills

LLM routing

self-learning

scheduled tasks

dreams

MCP

A2A

least privilege

From abstract "agent" talk to a working model for how agentic systems are **designed, governed, and extended.**



rockbot.dev · [github.com/MarimerLLC/rockbot](https://github.com/MarimerLLC/rockbot) · [blog.lhotka.net](https://blog.lhotka.net)